Chapitre 1 : Algèbre linéaire

Master IASC

Année universitaire 2026–2027

Plan du chapitre

1. Espaces vectoriels, bases et transformations linéaires
2. Déterminants
3. Valeurs propres et vecteurs propres
4. Diagonalisation
5. Réduction de dimension et projections : l'ACP

Pourquoi l'algèbre linéaire en IA ?

L'algèbre linéaire est le langage natif de l'apprentissage automatique :

- une **image** 28×28 en niveaux de gris $=$ un vecteur de $\mathbb{R}^{784}$;
- un **document texte** $=$ un vecteur TF-IDF ou un *embedding* de $\mathbb{R}^{768}$;
- un **réseau de neurones** $=$ une composition de transformations linéaires et de non-linéarités ;
- un **jeu de données** $=$ une matrice $X \in \mathbb{R}^{n \times d}$ (n individus, d variables).

Dans la pratique

NumPy, PyTorch et TensorFlow ne font (presque) que des produits matriciels optimisés sur GPU. Comprendre l'algèbre linéaire, c'est comprendre ce que calcule votre modèle.

Pourquoi l'algèbre linéaire en IA ?

L'algèbre linéaire est le langage natif de l'apprentissage automatique :

- une **image** 28×28 en niveaux de gris $=$ un vecteur de $\mathbb{R}^{784}$;
- un **document texte** $=$ un vecteur TF-IDF ou un *embedding* de $\mathbb{R}^{768}$;
- un **réseau de neurones** $=$ une composition de transformations linéaires et de non-linéarités ;
- un **jeu de données** $=$ une matrice $X \in \mathbb{R}^{n \times d}$ (n individus, d variables).

Dans la pratique

NumPy, PyTorch et TensorFlow ne font (presque) que des produits matriciels optimisés sur GPU. Comprendre l'algèbre linéaire, c'est comprendre ce que calcule votre modèle.

Plan du chapitre

1. Espaces vectoriels, bases et transformations linéaires
2. Déterminants
3. Valeurs propres et vecteurs propres
4. Diagonalisation
5. Réduction de dimension et projections : l'ACP

Espace vectoriel : définition

Définition — Espace vectoriel

Un ensemble E muni d'une addition $+$ et d'une multiplication par un scalaire $\cdot$ est un **espace vectoriel** sur $\mathbb{R}$ si, pour tous $u, v, w \in E$ et $\lambda, \mu \in \mathbb{R}$:

1. $(E, +)$ est un groupe commutatif (associativité, élément neutre 0_E, opposé) ;
2. $\lambda \cdot (u + v) = \lambda u + \lambda v, \quad (\lambda + \mu)u = \lambda u + \mu u$;
3. $\lambda(\mu u) = (\lambda\mu)u, \quad 1 \cdot u = u$.

- Exemples : $\mathbb{R}^d$, l'ensemble des matrices $\mathbb{R}^{n \times d}$, les polynômes $\mathbb{R}[X]$, les fonctions $f : \mathbb{R} \to \mathbb{R}$.
- Les éléments de E sont appelés **vecteurs**, ceux de $\mathbb{R}$ **scalaires**.

Espace vectoriel : définition

Définition — Espace vectoriel

Un ensemble E muni d'une addition $+$ et d'une multiplication par un scalaire $\cdot$ est un **espace vectoriel** sur $\mathbb{R}$ si, pour tous $u, v, w \in E$ et $\lambda, \mu \in \mathbb{R}$:

1. $(E, +)$ est un groupe commutatif (associativité, élément neutre 0_E, opposé) ;
2. $\lambda \cdot (u + v) = \lambda u + \lambda v, \quad (\lambda + \mu)u = \lambda u + \mu u$;
3. $\lambda(\mu u) = (\lambda\mu)u, \quad 1 \cdot u = u$.

- Exemples : $\mathbb{R}^d$, l'ensemble des matrices $\mathbb{R}^{n \times d}$, les polynômes $\mathbb{R}[X]$, les fonctions $f : \mathbb{R} \to \mathbb{R}$.
- Les éléments de E sont appelés **vecteurs**, ceux de $\mathbb{R}$ **scalaires**.

Les données sont des vecteurs

Exemple réel — représentations vectorielles

- **Vision** : image MNIST $28 \times 28 \to x \in \mathbb{R}^{784}$, chaque composante = intensité d'un pixel.
- **NLP** : mot $\to$ *word embedding* $w \in \mathbb{R}^{300}$ (word2vec, GloVe) ; célèbre arithmétique vectorielle : $w_{\text{roi}} - w_{\text{homme}} + w_{\text{femme}} \approx w_{\text{reine}}$.
- **Systèmes de recommandation** : utilisateur et film $\to$ vecteurs latents $u, v \in \mathbb{R}^k$, score prédit $= \langle u, v \rangle$.
- **Audio** : 1 seconde de son échantillonné à 16 kHz $\to x \in \mathbb{R}^{16000}$.

À retenir

Toute donnée numérisée vit dans un espace vectoriel : les opérations de l'IA sont des opérations vectorielles.

Les données sont des vecteurs

Exemple réel — représentations vectorielles

- **Vision** : image MNIST $28 \times 28 \to x \in \mathbb{R}^{784}$, chaque composante = intensité d'un pixel.
- **NLP** : mot $\to$ *word embedding* $w \in \mathbb{R}^{300}$ (word2vec, GloVe) ; célèbre arithmétique vectorielle : $w_{\text{roi}} - w_{\text{homme}} + w_{\text{femme}} \approx w_{\text{reine}}$.
- **Systèmes de recommandation** : utilisateur et film $\to$ vecteurs latents $u, v \in \mathbb{R}^k$, score prédit $= \langle u, v \rangle$.
- **Audio** : 1 seconde de son échantillonné à 16 kHz $\to x \in \mathbb{R}^{16000}$.

À retenir

Toute donnée numérisée vit dans un espace vectoriel : les opérations de l'IA sont des opérations vectorielles.

Les données sont des vecteurs

Exemple réel — représentations vectorielles

- **Vision** : image MNIST $28 \times 28 \to x \in \mathbb{R}^{784}$, chaque composante = intensité d'un pixel.
- **NLP** : mot $\to$ *word embedding* $w \in \mathbb{R}^{300}$ (word2vec, GloVe) ; célèbre arithmétique vectorielle : $w_{\text{roi}} - w_{\text{homme}} + w_{\text{femme}} \approx w_{\text{reine}}$.
- **Systèmes de recommandation** : utilisateur et film $\to$ vecteurs latents $u, v \in \mathbb{R}^k$, score prédit $= \langle u, v \rangle$.
- **Audio** : 1 seconde de son échantillonné à 16 kHz $\to x \in \mathbb{R}^{16000}$.

À retenir

Toute donnée numérisée vit dans un espace vectoriel : les opérations de l'IA sont des opérations vectorielles.

Les données sont des vecteurs

Exemple réel — représentations vectorielles

- **Vision** : image MNIST $28 \times 28 \to x \in \mathbb{R}^{784}$, chaque composante = intensité d'un pixel.
- **NLP** : mot $\to$ *word embedding* $w \in \mathbb{R}^{300}$ (word2vec, GloVe) ; célèbre arithmétique vectorielle : $w_{\text{roi}} - w_{\text{homme}} + w_{\text{femme}} \approx w_{\text{reine}}$.
- **Systèmes de recommandation** : utilisateur et film $\to$ vecteurs latents $u, v \in \mathbb{R}^k$, score prédit $= \langle u, v \rangle$.
- **Audio** : 1 seconde de son échantillonné à 16 kHz $\to x \in \mathbb{R}^{16000}$.

À retenir

Toute donnée numérisée vit dans un espace vectoriel : les opérations de l'IA sont des opérations vectorielles.

Les données sont des vecteurs

Exemple réel — représentations vectorielles

- **Vision** : image MNIST $28 \times 28 \to x \in \mathbb{R}^{784}$, chaque composante = intensité d'un pixel.
- **NLP** : mot $\to$ *word embedding* $w \in \mathbb{R}^{300}$ (word2vec, GloVe) ; célèbre arithmétique vectorielle : $w_{\text{roi}} - w_{\text{homme}} + w_{\text{femme}} \approx w_{\text{reine}}$.
- **Systèmes de recommandation** : utilisateur et film $\to$ vecteurs latents $u, v \in \mathbb{R}^k$, score prédit $= \langle u, v \rangle$.
- **Audio** : 1 seconde de son échantillonné à 16 kHz $\to x \in \mathbb{R}^{16000}$.

À retenir

Toute donnée numérisée vit dans un espace vectoriel : les opérations de l'IA sont des opérations vectorielles.

Les données sont des vecteurs

Exemple réel — représentations vectorielles

- **Vision** : image MNIST $28 \times 28 \to x \in \mathbb{R}^{784}$, chaque composante $=$ intensité d'un pixel.
- **NLP** : mot $\to$ *word embedding* $w \in \mathbb{R}^{300}$ (word2vec, GloVe) ; célèbre arithmétique vectorielle : $w_{\text{roi}} - w_{\text{homme}} + w_{\text{femme}} \approx w_{\text{reine}}$.
- **Systèmes de recommandation** : utilisateur et film $\to$ vecteurs latents $u, v \in \mathbb{R}^k$, score prédit $= \langle u, v \rangle$.
- **Audio** : 1 seconde de son échantillonné à 16 kHz $\to x \in \mathbb{R}^{16000}$.

À retenir

Toute donnée numérisée vit dans un espace vectoriel : les opérations de l'IA sont des opérations vectorielles.

Sous-espaces vectoriels

Définition — Sous-espace vectoriel

$F \subseteq E$ est un **sous-espace vectoriel** de E si : $0_E \in F$ et $\forall u, v \in F,\ \forall \lambda \in \mathbb{R} :\ u + \lambda v \in F$.

- Droites et plans passant par l'origine dans $\mathbb{R}^3$;
- L'ensemble des solutions de $Ax = 0$ (noyau) ;
- $\mathrm{Vect}(v_1, \ldots, v_p) = \{\sum_i \lambda_i v_i\}$: le sous-espace **engendré**.

Hypothèse du sous-espace

En pratique, les images " naturelles " de $\mathbb{R}^{784}$ vivent près d'un sous-espace (ou d'une variété) de dimension bien plus faible : c'est ce qui rend la **réduction de dimension** possible (ACP, §5).

Familles libres, génératrices, bases

Définitions

Soit $(v_1, \dots, v_p)$ une famille de vecteurs de E :

- **libre** si $\sum_i \lambda_i v_i = 0 \Rightarrow \lambda_1 = \cdots = \lambda_p = 0$;
- **génératrice** si $\mathrm{Vect}(v_1, \dots, v_p) = E$;
- **base** si elle est libre *et* génératrice.

Théorème — Dimension

Toutes les bases de E ont le même cardinal, appelé **dimension** de E, notée $\dim E$. Dans une base $\mathcal{B} = (e_1, \dots, e_d)$, tout $x \in E$ s'écrit de façon *unique* $x = \sum_{i=1}^{d} x_i e_i$: les x_i sont les **coordonnées** de x dans $\mathcal{B}$.

- Base canonique de $\mathbb{R}^d$: $e_i = (0, \dots, 1, \dots, 0)$ (1 en position i).

Bases en informatique : le codage one-hot

Exemple réel — encodage one-hot

Pour un vocabulaire $\{\text{chat}, \text{chien}, \text{oiseau}\}$, chaque mot est codé par un vecteur de la base canonique de $\mathbb{R}^3$:

$$\text{chat} = \begin{pmatrix} 1 \\ 0 \\ 0 \end{pmatrix}, \quad \text{chien} = \begin{pmatrix} 0 \\ 1 \\ 0 \end{pmatrix}, \quad \text{oiseau} = \begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix}.$$

C'est l'entrée standard des modèles de classification (variables catégorielles, tokens en NLP).

À retenir

Changer de base = changer de **représentation** des données, sans changer l'information. Beaucoup d'algorithmes d'IA (ACP, embeddings) cherchent précisément une " meilleure base ".

Bases en informatique : le codage one-hot

Exemple réel — encodage one-hot

Pour un vocabulaire $\{\text{chat}, \text{chien}, \text{oiseau}\}$, chaque mot est codé par un vecteur de la base canonique de $\mathbb{R}^3$:

$$\text{chat} = \begin{pmatrix} 1 \\ 0 \\ 0 \end{pmatrix}, \quad \text{chien} = \begin{pmatrix} 0 \\ 1 \\ 0 \end{pmatrix}, \quad \text{oiseau} = \begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix}.$$

C'est l'entrée standard des modèles de classification (variables catégorielles, tokens en NLP).

À retenir

Changer de base = changer de **représentation** des données, sans changer l'information. Beaucoup d'algorithmes d'IA (ACP, embeddings) cherchent précisément une " meilleure base ".

Bases en informatique : le codage one-hot

Exemple réel — encodage one-hot

Pour un vocabulaire $\{\text{chat}, \text{chien}, \text{oiseau}\}$, chaque mot est codé par un vecteur de la base canonique de $\mathbb{R}^3$:

$$\text{chat} = \begin{pmatrix} 1 \\ 0 \\ 0 \end{pmatrix}, \quad \text{chien} = \begin{pmatrix} 0 \\ 1 \\ 0 \end{pmatrix}, \quad \text{oiseau} = \begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix}.$$

C'est l'entrée standard des modèles de classification (variables catégorielles, tokens en NLP).

À retenir

Changer de base $=$ changer de **représentation** des données, sans changer l'information. Beaucoup d'algorithmes d'IA (ACP, embeddings) cherchent précisément une " meilleure base ".

Applications linéaires

Définition — Application linéaire

$f : E \to F$ est **linéaire** si $\forall u, v \in E,\ \forall \lambda \in \mathbb{R}$:

$$f(u+v) = f(u) + f(v) \qquad \text{et} \qquad f(\lambda u) = \lambda f(u).$$

- **Noyau** : $\ker f = \{x \in E : f(x) = 0\}$ (sous-espace de E) ;
- **Image** : $\operatorname{Im} f = \{f(x) : x \in E\}$ (sous-espace de F) ;
- f injective $\iff \ker f = \{0\}$.

Théorème du rang

Si $\dim E < \infty$: $\dim E = \dim \ker f + \operatorname{rg} f$, où $\operatorname{rg} f = \dim \operatorname{Im} f$.

Matrice d'une application linéaire

Soit $f : \mathbb{R}^d \to \mathbb{R}^n$ linéaire, $\mathcal{B} = (e_1, \ldots, e_d)$ la base canonique.

Représentation matricielle

La matrice $A \in \mathbb{R}^{n \times d}$ dont la j-ième colonne est $f(e_j)$ vérifie :

$$f(x) = Ax \qquad \text{pour tout } x \in \mathbb{R}^d.$$

La composition d'applications correspond au **produit matriciel** : $M_{g \circ f} = M_g \, M_f$.

Exemple réel — couche dense d'un réseau de neurones

Une couche `Dense(512, 128)` calcule $z = Wx + b$ avec $W \in \mathbb{R}^{128 \times 512}$: c'est une application *affine* (linéaire + translation). Le réseau complet :

$$\hat{y} = W_3 \, \sigma\big(W_2 \, \sigma(W_1 x + b_1) + b_2\big) + b_3.$$

Sans les non-linéarités σ, la composition resterait… une seule application affine !

Transformations linéaires en Python

```python
import numpy as np

# Une "couche" lineaire : R^3 -> R^2
W = np.array([[1., 0., -1.],
              [2., 1.,  0.]])
x = np.array([1., 2., 3.])
z = W @ x                  # produit matrice-vecteur : [-2., 4.]

# Composition = produit matriciel
W2 = np.array([[1., 1.],
               [0., 3.]])
y = W2 @ (W @ x)           # equivalent a (W2 @ W) @ x
print(np.allclose(y, (W2 @ W) @ x))   # True : associativite
```

- `@` : produit matriciel NumPy ; sur GPU, la même opération traite des *lots* entiers de données.

Changement de base

Soit $\mathcal{B}$ et $\mathcal{B}'$ deux bases de $\mathbb{R}^d$, et P la matrice **de passage** (colonnes $=$ vecteurs de $\mathcal{B}'$ exprimés dans $\mathcal{B}$).

Formules de changement de base

- Coordonnées : $x_{\mathcal{B}} = P\, x_{\mathcal{B}'}$, donc $x_{\mathcal{B}'} = P^{-1} x_{\mathcal{B}}$;
- Matrice d'un endomorphisme : $A' = P^{-1}AP$ (matrices **semblables**).

Intuition IA

L'ACP (fin de ce chapitre) est exactement un changement de base *orthonormée* choisi pour que les nouvelles coordonnées soient décorrélées et ordonnées par importance.

Produit scalaire, norme, similarité

Produit scalaire euclidien et norme

$$\langle x, y \rangle = x^{\mathsf{T}} y = \sum_{i=1}^{d} x_i y_i, \qquad \|x\|_2 = \sqrt{\langle x, x \rangle}, \qquad \cos(x, y) = \frac{\langle x, y \rangle}{\|x\|_2 \, \|y\|_2}.$$

Exemple réel — similarité cosinus

Moteurs de recherche sémantique et RAG : un texte est encodé en vecteur $\mathbb{R}^{768}$; les documents les plus “ proches ” d’une requête sont ceux qui maximisent le **cosinus**. C’est aussi la base des recommandations “ articles similaires ”.

- Deux vecteurs sont **orthogonaux** si $\langle x, y \rangle = 0$.
- Une base est **orthonormée** si ses vecteurs sont unitaires et deux à deux orthogonaux.

Plan du chapitre

1 Espaces vectoriels, bases et transformations linéaires

2 Déterminants

3 Valeurs propres et vecteurs propres

4 Diagonalisation

5 Réduction de dimension et projections : l'ACP

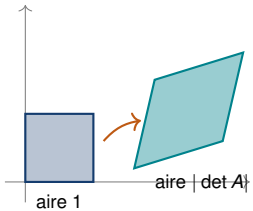
Déterminant : définition et intuition

Déterminant (ordre 2 et 3)

$$\det \begin{pmatrix} a & b \\ c & d \end{pmatrix} = ad - bc, \qquad \det A \text{ pour } A \in \mathbb{R}^{3\times 3} \text{ : règle de Sarrus ou développement.}$$

Interprétation géométrique :

- $|\det A| =$ facteur de dilatation des **volumes** par $x \mapsto Ax$;
- $\det A = 0 \iff A$ " écrase " l'espace sur un sous-espace strict ;
- signe $=$ préservation ou non de l'orientation.



Propriétés fondamentales du déterminant

Propriétés

Pour $A, B \in \mathbb{R}^{d \times d}$:

1. $\det(AB) = \det A \cdot \det B$; $\quad \det(A^{\mathsf{T}}) = \det A$;
2. $\det(\lambda A) = \lambda^d \det A$;
3. A **inversible** $\iff \det A \neq 0$, et alors $\det(A^{-1}) = 1/\det A$;
4. det d'une matrice triangulaire $=$ produit des éléments diagonaux ;
5. échanger deux lignes change le signe ; ajouter à une ligne un multiple d'une autre ne change rien.

À retenir

En pratique on ne calcule *jamais* un déterminant par développement en cofacteurs ($\mathcal{O}(d!)$) mais par **pivot de Gauss** ($\mathcal{O}(d^3)$) — c'est ce que fait `numpy.linalg.det`.

Développement en cofacteurs

Développement selon une ligne

$$\det A = \sum_{j=1}^{d} (-1)^{i+j}\, a_{ij}\, \det A_{ij},$$

où A_{ij} est la matrice A privée de la ligne i et de la colonne j (**mineur**).

Exemple :

$$\det \begin{pmatrix} 2 & 1 & 0 \\ 1 & 3 & 1 \\ 0 & 1 & 2 \end{pmatrix} = 2 \det \begin{pmatrix} 3 & 1 \\ 1 & 2 \end{pmatrix} - 1 \det \begin{pmatrix} 1 & 1 \\ 0 & 2 \end{pmatrix} + 0 = 2(6-1) - 1(2-0) = 8.$$

Déterminants en IA : où les rencontre-t-on ?

Exemple réel 1 — Loi normale multivariée

La densité de $\mathcal{N}(\mu, \Sigma)$ sur $\mathbb{R}^d$ fait intervenir $\det \Sigma$:

$$f(x) = \frac{1}{(2\pi)^{d/2}\,(\det \Sigma)^{1/2}} \exp\Big(-\tfrac{1}{2}(x-\mu)^{\mathsf{T}} \Sigma^{-1} (x-\mu) \Big).$$

Utilisée dans les mélanges gaussiens (GMM), la détection d'anomalies, les filtres de Kalman.

Exemple réel 2 — Normalizing flows (modèles génératifs)

Pour transformer une loi simple en loi complexe via $z \mapsto g(z)$, la formule du changement de variables exige le **jacobien** : $p_X(x) = p_Z(z)\,\big|\det J_g(z)\big|^{-1}$. Les architectures (RealNVP, Glow) sont conçues pour que ce déterminant soit calculable en $\mathcal{O}(d)$!

Déterminant et inversibilité en Python

```python
import numpy as np

A = np.array([[2., 1., 0.],
              [1., 3., 1.],
              [0., 1., 2.]])
print(np.linalg.det(A))         # 8.0  -> A est inversible
x = np.linalg.solve(A, np.array([1., 2., 3.]))  # resout Ax = b

# Matrice singuliere : colonnes liees
B = np.array([[1., 2.],
              [2., 4.]])        # colonne2 = 2 * colonne1
print(np.linalg.det(B))         # 0.0  -> non inversible
```

À retenir

Test numérique d'inversibilité : ne comparez pas det à 0 exactement, regardez plutôt le **conditionnement** (`np.linalg.cond`) — crucial pour la stabilité des algorithmes de ML.

Plan du chapitre

1. Espaces vectoriels, bases et transformations linéaires
2. Déterminants
3. Valeurs propres et vecteurs propres
4. Diagonalisation
5. Réduction de dimension et projections : l'ACP

Valeurs propres : définition

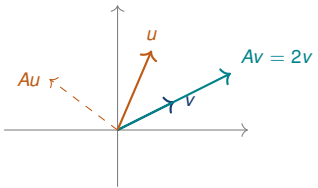
Définition — Éléments propres

Soit $A \in \mathbb{R}^{d \times d}$. Un scalaire λ est une **valeur propre** de A s'il existe $v \neq 0$ tel que

$$Av = \lambda v.$$

v est un **vecteur propre** associé à λ. Le sous-espace $E_\lambda = \ker(A - \lambda I)$ est le **sous-espace propre**.

Intuition : les vecteurs propres sont les directions que A ne fait qu'*étirer ou comprimer* (facteur λ), sans les faire tourner.



Polynôme caractéristique

Caractérisation

λ est valeur propre de $A \iff \det(A - \lambda I) = 0$. Le polynôme $\chi_A(\lambda) = \det(A - \lambda I)$, de degré d, est le **polynôme caractéristique** de A.

Exemple : $A = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}$

$$\chi_A(\lambda) = \det \begin{pmatrix} 2-\lambda & 1 \\ 1 & 2-\lambda \end{pmatrix} = (2-\lambda)^2 - 1 = (\lambda - 1)(\lambda - 3).$$

Valeurs propres : $\lambda_1 = 3$ avec $v_1 = \frac{1}{\sqrt{2}}\binom{1}{1}$, $\quad \lambda_2 = 1$ avec $v_2 = \frac{1}{\sqrt{2}}\binom{1}{-1}$.

- Propriétés utiles : $\mathrm{tr} A = \sum_i \lambda_i, \quad \det A = \prod_i \lambda_i$.

Exemple réel : PageRank de Google

Le web comme une matrice

Soit $P \in \mathbb{R}^{n \times n}$ la matrice de transition du " surfeur aléatoire " sur le graphe du web : $P_{ij} =$ probabilité d'aller de la page j à la page i. Le score PageRank π est le vecteur (positif, de somme 1) vérifiant :

$$\pi = G\pi, \qquad G = \alpha P + (1-\alpha)\tfrac{1}{n}\mathbf{1}\mathbf{1}^{\mathsf{T}}, \quad \alpha \approx 0{,}85.$$

π est le **vecteur propre** de G associé à la valeur propre 1 !

- Calculé par la **méthode de la puissance** : $\pi^{(t+1)} = G\pi^{(t)}$, qui converge vers le vecteur propre dominant.
- Même principe : centralité spectrale dans les réseaux sociaux, HITS, spectral clustering.

PageRank en 10 lignes de NumPy

```python
import numpy as np

# Petit web de 4 pages : liens j -> i (colonnes stochastiques)
P = np.array([[0. , 0.5, 0. , 0. ],
              [0.5, 0. , 0. , 1. ],
              [0.5, 0. , 0. , 0. ],
              [0. , 0.5, 1. , 0. ]])
n, alpha = 4, 0.85
G = alpha * P + (1 - alpha) / n * np.ones((n, n))

pi = np.ones(n) / n
for _ in range(100):          # methode de la puissance
    pi = G @ pi
print(pi.round(3))            # scores PageRank
```

- Google résout ce problème sur un graphe à *des centaines de milliards* de pages (matrices creuses + calcul distribué).

Multiplicités et spectre

Multiplicités

- **Multiplicité algébrique** $m_a(\lambda)$: ordre de λ comme racine de χ_A ;
- **Multiplicité géométrique** $m_g(\lambda) = \dim E_\lambda$;
- Toujours : $1 \leq m_g(\lambda) \leq m_a(\lambda)$.

Indépendance des vecteurs propres

Des vecteurs propres associés à des valeurs propres *distinctes* sont linéairement indépendants.

- L'ensemble des valeurs propres est le **spectre** de A, noté $\mathrm{Sp}(A)$.
- Le **rayon spectral** $\rho(A) = \max_i |\lambda_i|$ contrôle le comportement de A^t (stabilité des systèmes dynamiques, convergence des itérations, explosion/évanouissement des gradients dans les RNN).

Plan du chapitre

1 Espaces vectoriels, bases et transformations linéaires

2 Déterminants

3 Valeurs propres et vecteurs propres

4 Diagonalisation

5 Réduction de dimension et projections : l'ACP

Diagonalisation : définition et critères

Matrice diagonalisable

$A \in \mathbb{R}^{d\times d}$ est **diagonalisable** s'il existe P inversible et D diagonale telles que

$$A = PDP^{-1}, \qquad D = \mathrm{diag}(\lambda_1, \ldots, \lambda_d).$$

Les colonnes de P sont des vecteurs propres, les λ_i les valeurs propres associées.

Critères

- A diagonalisable $\iff$ il existe une base de vecteurs propres $\iff$ $\sum_\lambda m_g(\lambda) = d$;
- **Condition suffisante** : d valeurs propres distinctes ;
- Contre-exemple : $\begin{pmatrix} 1 & 1 \\ 0 & 1 \end{pmatrix}$ n'est pas diagonalisable ($m_a(1) = 2$, $m_g(1) = 1$).

Pourquoi diagonaliser ? Les puissances de matrices

Calcul de A^t

Si $A = PDP^{-1}$, alors

$$A^t = PD^tP^{-1} = P\,\mathrm{diag}(\lambda_1^t, \ldots, \lambda_d^t)\,P^{-1}.$$

Le comportement à long terme de $A^t x$ est dicté par les $|\lambda_i|$: divergence si $|\lambda| > 1$, extinction si $|\lambda| < 1$.

Exemples réels

- **Chaînes de Markov** : distribution après t pas $= \pi^{(0)} P^t$; la convergence vers le régime stationnaire est gouvernée par la 2e valeur propre.
- **RNN / gradients** : la rétropropagation multiplie t fois par W^T ; $\rho(W) > 1 \Rightarrow$ gradients qui explosent, $\rho(W) < 1 \Rightarrow$ gradients qui s'évanouissent.
- **Graphes** : $(A^t)_{ij}$ compte les chemins de longueur t entre i et j.

Exemple complet de diagonalisation

Reprenons $A = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}$, avec $\lambda_1 = 3$, $\lambda_2 = 1$:

$$P = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}, \qquad D = \begin{pmatrix} 3 & 0 \\ 0 & 1 \end{pmatrix}, \qquad A = PDP^{-1} \text{ (ici } P^{-1} = P^{\mathsf{T}}).$$

Alors :

$$A^{10} = P \begin{pmatrix} 3^{10} & 0 \\ 0 & 1 \end{pmatrix} P^{\mathsf{T}} = \frac{1}{2} \begin{pmatrix} 3^{10}+1 & 3^{10}-1 \\ 3^{10}-1 & 3^{10}+1 \end{pmatrix}.$$

- Coût : une diagonalisation $\mathcal{O}(d^3)$ puis A^t en $\mathcal{O}(d^2)$ pour tout t, au lieu de $t-1$ produits matriciels.

Le théorème spectral : le cas des matrices symétriques

Théorème spectral

Toute matrice **symétrique réelle** $A = A^\top$ est diagonalisable dans une **base orthonormée** :

$$A = Q\Lambda Q^\top, \qquad Q^\top Q = I, \quad \Lambda = \mathrm{diag}(\lambda_1, \ldots, \lambda_d),$$

et toutes ses valeurs propres sont *réelles*.

Pourquoi c'est central en IA

Les matrices de **covariance** $\Sigma = \frac{1}{n} X_c^\top X_c$, les matrices de Gram K (noyaux), les hessiennes $\nabla^2 f$ sont symétriques : le théorème spectral s'applique toujours.

- Σ est de plus **semi-définie positive** : $\lambda_i \geq 0$;
- les valeurs propres de la hessienne indiquent la courbure de la fonction de perte (minimum, col, plateau) — au cœur de l'optimisation des réseaux de neurones.

Plan du chapitre

1 Espaces vectoriels, bases et transformations linéaires

2 Déterminants

3 Valeurs propres et vecteurs propres

4 Diagonalisation

5 Réduction de dimension et projections : l'ACP

Projections orthogonales

Projection orthogonale sur un sous-espace

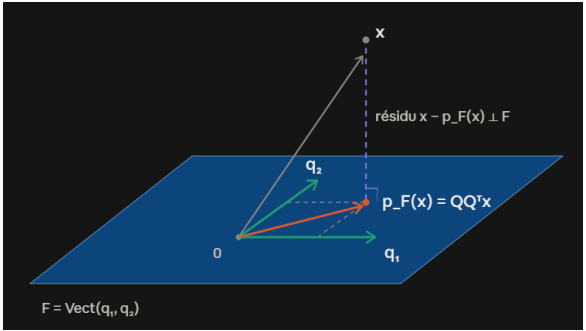
Soit $F \subseteq \mathbb{R}^d$ un sous-espace muni d'une base orthonormée $(q_1, \ldots, q_k)$, et $Q = [q_1 | \cdots | q_k] \in \mathbb{R}^{d \times k}$. La **projection orthogonale** de x sur F est

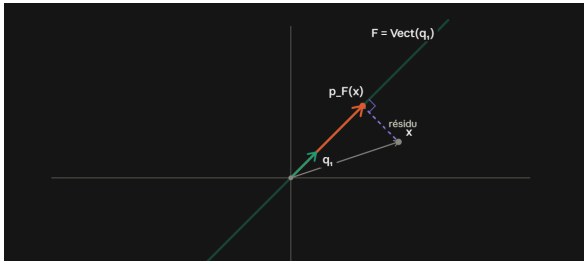
$$p_F(x) = QQ^{\mathsf{T}}x = \sum_{i=1}^{k} \langle x, q_i \rangle \, q_i.$$

C'est le point de F *le plus proche* de x : $p_F(x) = \arg\min_{y \in F} \|x - y\|_2$.

- $x - p_F(x)$ est orthogonal à F (théorème de Pythagore : $\|x\|^2 = \|p_F(x)\|^2 + \|x - p_F(x)\|^2$).
- Idée clé de la réduction de dimension : remplacer $x \in \mathbb{R}^d$ par ses k coordonnées $Q^{\mathsf{T}}x \in \mathbb{R}^k$, avec $k \ll d$, en perdant le moins d'information possible.

Projection orthogonale : illustration





- La projection $p_F(x)$ est *dans* F ; le résidu $x - p_F(x)$ est *orthogonal* à F.

La malédiction de la dimension

Pourquoi réduire la dimension ?

- **Coût** : entraîner sur $\mathbb{R}^{10^6}$ est lent et gourmand en mémoire ; p. ex. une photo de téléphone (12 Mpx $\times$ 3 couleurs) est un vecteur de 36×10^6 nombres ; pour la classer (“ plage ”, “ visage ”...), quelques centaines de caractéristiques suffisent, soit 10^5 fois moins de nombres.
- **Statistique** : avec 2 critères (taille, poids), on trouve facilement un élève qui nous ressemble ; avec 100 critères, tous paraissent aussi différents, et il faut exponentiellement plus de données (*curse of dimensionality*) ;
- **Visualisation** : impossible de “ voir ” au-delà de 3D ; ex. *Iris* ($\mathbb{R}^4$, 3 espèces) devient lisible une fois projetée en 2D.

- Solution linéaire de référence : **l’Analyse en Composantes Principales (ACP)**, *Principal Component Analysis (PCA)* en anglais.

La malédiction de la dimension

Pourquoi réduire la dimension ?

- **Coût** : entraîner sur $\mathbb{R}^{10^6}$ est lent et gourmand en mémoire ; p. ex. une photo de téléphone (12 Mpx $\times$ 3 couleurs) est un vecteur de 36×10^6 nombres ; pour la classer (“ plage ”, “ visage ”...), quelques centaines de caractéristiques suffisent, soit 10^5 fois moins de nombres.
- **Statistique** : avec 2 critères (taille, poids), on trouve facilement un élève qui nous ressemble ; avec 100 critères, tous paraissent aussi différents, et il faut exponentiellement plus de données (*curse of dimensionality*) ;
- **Visualisation** : impossible de “ voir ” au-delà de 3D ; ex. *Iris* ($\mathbb{R}^4$, 3 espèces) devient lisible une fois projetée en 2D.

- Solution linéaire de référence : **l’Analyse en Composantes Principales (ACP)**, *Principal Component Analysis (PCA)* en anglais.

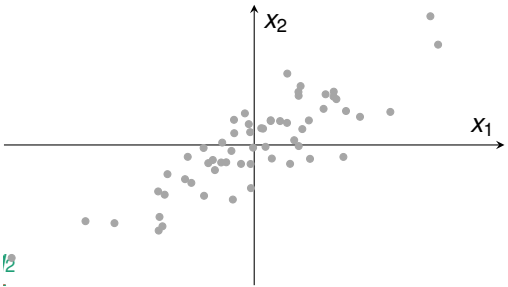
La malédiction de la dimension

Pourquoi réduire la dimension ?

- **Coût** : entraîner sur $\mathbb{R}^{10^6}$ est lent et gourmand en mémoire ; p. ex. une photo de téléphone (12 Mpx $\times$ 3 couleurs) est un vecteur de 36×10^6 nombres ; pour la classer (“ plage ”, “ visage ”...), quelques centaines de caractéristiques suffisent, soit 10^5 fois moins de nombres.
- **Statistique** : avec 2 critères (taille, poids), on trouve facilement un élève qui nous ressemble ; avec 100 critères, tous paraissent aussi différents, et il faut exponentiellement plus de données (*curse of dimensionality*) ;
- **Visualisation** : impossible de “ voir ” au-delà de 3D ; ex. *Iris* ($\mathbb{R}^4$, 3 espèces) devient lisible une fois projetée en 2D.

- Solution linéaire de référence : **l’Analyse en Composantes Principales (ACP)**, *Principal Component Analysis (PCA)* en anglais.

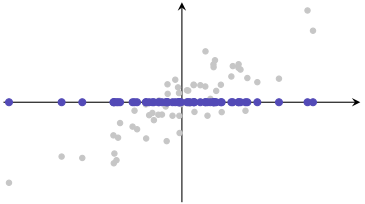
ACP : l'idée en une image



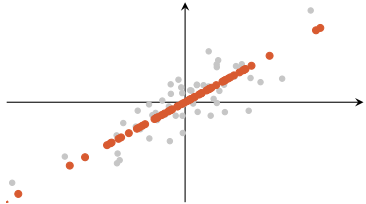
- Nuage de points centré dans $\mathbb{R}^2$.
- q_1 : direction où le nuage est **le plus étalé**.
- $q_2 \perp q_1$: ce qui reste.
- Projeter sur q_1 (pointillés) : on passe de 2 nombres à 1, en perdant très peu.

Bon axe ou mauvais axe ?

Projection sur e_1 (axe x_1)



Projection sur q_1



Direction u	e_1	q_1	q_2
Variance projetée $u^T \Sigma u$	2.33	**2.98**	0.17
% de la variance totale (= 3.15)	74 %	**95 %**	5 %

Plus les points projetés sont étalés, plus on garde d'information.
Chaque colonne est calculée séparément : les % ne s'additionnent pas.

ACP : formulation

Données : $X \in \mathbb{R}^{n \times d}$, lignes x_i *centrées* ($\bar{x} = 0$; on centre sinon).

Deux formulations équivalentes

Chercher une base orthonormée $(q_1, \dots, q_k)$ qui :

- **maximise la variance projetée** : $q_1 = \arg\max_{\|q\|=1} \frac{1}{n} \sum_{i=1}^{n} \langle x_i, q \rangle^2 = \arg\max_{\|q\|=1} q^\top \Sigma q$,
  puis $q_2 \perp q_1$ de variance maximale, etc. ;
- ou, de façon équivalente, **minimise l'erreur de reconstruction** : $\min_{Q^\top Q = I_k} \frac{1}{n} \sum_{i=1}^{n} \left\| x_i - Q Q^\top x_i \right\|_2^2$,

où $\Sigma = \frac{1}{n} X^\top X \in \mathbb{R}^{d \times d}$ est la matrice de **covariance empirique**.

ACP : solution par le théorème spectral

Solution de l'ACP

Soit $\Sigma = Q\Lambda Q^{\mathsf{T}}$ la décomposition spectrale de la covariance, avec $\lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_d \geq 0$.

- Les **composantes principales** sont les vecteurs propres $q_1, \ldots, q_k$ associés aux k plus grandes valeurs propres ;
- La variance capturée par q_j est λ_j ; la **part de variance expliquée** par les k premières composantes est
$$\frac{\lambda_1 + \cdots + \lambda_k}{\lambda_1 + \cdots + \lambda_d}.$$
- Avec $Q_k = [q_1 | \cdots | q_k] \in \mathbb{R}^{d \times k}$: nouvelles coordonnées (*scores*) $Z = XQ_k \in \mathbb{R}^{n \times k}$; reconstruction : $\hat{X} = ZQ_k^{\mathsf{T}}$.

Exemple réel : la base *Iris*

150 fleurs, 3 espèces, 4 mesures (cm) $\Rightarrow x \in \mathbb{R}^4$, données centrées.
Les 4 variables ont la même unité (cm) : on ne les réduit pas (ACP *non normée*).

Variables d'origine

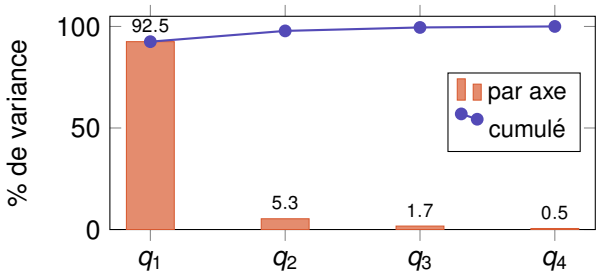
Variable	Variance
Long. sépale	0.68
Larg. sépale	0.19
Long. pétale	3.10
Larg. pétale	0.58
Total	4.54

Axes de l'ACP

Axe	λ_j	%	% cumulé
q_1	4.20	92.5	**92.5**
q_2	0.24	5.3	**97.8**
q_3	0.08	1.7	99.5
q_4	0.02	0.5	100
Total	4.54	100	

- La variance totale est la même (4.54) : l'ACP ne fait que tourner les axes.
- Mais elle la concentre : 2 axes sur 4 gardent **97.8 %** de l'information.

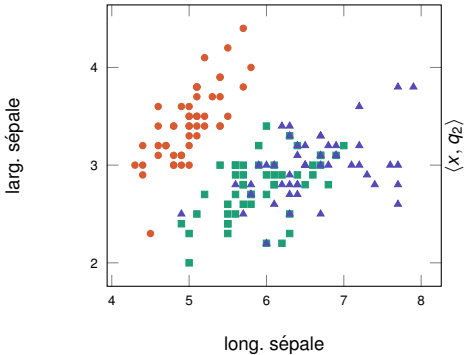
Choisir le nombre de composantes k



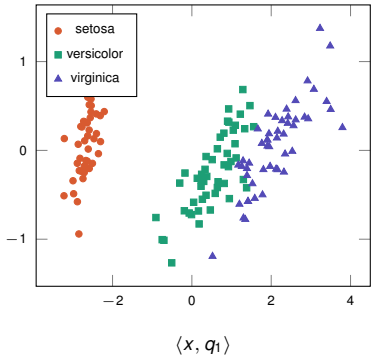
- **Seuil** : plus petit k tel que le cumul dépasse 90–95 % ; ici $k = 2$: $\mathbb{R}^4 \to \mathbb{R}^2$;
- **Coude** : on s'arrête là où la courbe des λ_j s'aplatit (ici après q_1) ;
- **Validation croisée** : le k qui donne la meilleure performance du modèle en aval.

Iris : avant et après l'ACP

2 variables d'origine



Projection sur (q_1, q_2)



À gauche, versicolor et virginica se mélangent. À droite, avec **2 nombres** par fleur, les espèces sont nettement mieux séparées le long de q_1.

Autre exemple réel

MNIST

Sur les chiffres manuscrits ($d = 784$), environ **87 composantes** capturent 90 % de la variance (154 pour 95 %) : compression $\times 9$ avant un classifieur k-NN ou SVM, avec un gain massif en temps de calcul.

- Autres usages : visualisation 2D de datasets, débruitage, décorrélation avant clustering, compression d'embeddings dans les bases vectorielles (RAG).

Autre exemple réel

MNIST

Sur les chiffres manuscrits ($d = 784$), environ **87 composantes** capturent 90 % de la variance (154 pour 95 %) : compression $\times 9$ avant un classifieur k-NN ou SVM, avec un gain massif en temps de calcul.

- Autres usages : visualisation 2D de datasets, débruitage, décorrélation avant clustering, compression d'embeddings dans les bases vectorielles (RAG).

Autre exemple réel

MNIST

Sur les chiffres manuscrits ($d = 784$), environ **87 composantes** capturent 90 % de la variance (154 pour 95 %) : compression $\times 9$ avant un classifieur k-NN ou SVM, avec un gain massif en temps de calcul.

- Autres usages : visualisation 2D de datasets, débruitage, décorrélation avant clustering, compression d'embeddings dans les bases vectorielles (RAG).

ACP avec scikit-learn

```python
import numpy as np
from sklearn.datasets import load_digits
from sklearn.decomposition import PCA

X, y = load_digits(return_X_y=True)      # 1797 images 8x8 -> R^64
pca = PCA(n_components=2)
Z = pca.fit_transform(X)                 # scores : (1797, 2)

print(pca.explained_variance_ratio_)     # part de variance de PC1, PC2
# Reconstruction approchee avec k composantes
pca30 = PCA(n_components=30).fit(X)
X_hat = pca30.inverse_transform(pca30.transform(X))
err = np.mean((X - X_hat) ** 2)          # erreur de reconstruction
```

- En interne : centrage, puis décomposition (SVD) — exactement la théorie de ce chapitre.
- Le TP 1 mettra en œuvre l'ACP *à la main* (covariance + `np.linalg.eigh`) puis la comparera à scikit-learn.

ACP avec scikit-learn

```python
import numpy as np
from sklearn.datasets import load_digits
from sklearn.decomposition import PCA

X, y = load_digits(return_X_y=True)      # 1797 images 8x8 -> R^64
pca = PCA(n_components=2)
Z = pca.fit_transform(X)                 # scores : (1797, 2)

print(pca.explained_variance_ratio_)     # part de variance de PC1, PC2
# Reconstruction approchee avec k composantes
pca30 = PCA(n_components=30).fit(X)
X_hat = pca30.inverse_transform(pca30.transform(X))
err = np.mean((X - X_hat) ** 2)          # erreur de reconstruction
```

- En interne : centrage, puis décomposition (SVD) — exactement la théorie de ce chapitre.
- Le TP 1 mettra en œuvre l'ACP *à la main* (covariance + `np.linalg.eigh`) puis la comparera à scikit-learn.

Limites de l'ACP et ouvertures

- **Linéarité** : l'ACP ne capture que des structures linéaires ; pour des variétés courbes $\rightarrow$ kernel PCA, t-SNE, UMAP, auto-encodeurs ;
- **Sensibilité à l'échelle** : standardiser les variables si elles ont des unités différentes (une variable en mètres vs en kilomètres dominerait la variance) ; inutile pour *Iris* (tout en cm) ;
- **Interprétabilité** : les composantes sont des combinaisons de toutes les variables ;
- **Variance $\neq$ pertinence** : la direction la plus variable n'est pas forcément la plus discriminante pour une tâche de classification (voir LDA).

Lien avec le deep learning

Un auto-encodeur *linéaire* à goulot de dimension k entraîné en erreur quadratique apprend exactement le sous-espace des k premières composantes principales : l'ACP est " l'ancêtre linéaire " des représentations apprises.

Limites de l'ACP et ouvertures

- **Linéarité** : l'ACP ne capture que des structures linéaires ; pour des variétés courbes $\rightarrow$ kernel PCA, t-SNE, UMAP, auto-encodeurs ;
- **Sensibilité à l'échelle** : standardiser les variables si elles ont des unités différentes (une variable en mètres vs en kilomètres dominerait la variance) ; inutile pour *Iris* (tout en cm) ;
- **Interprétabilité** : les composantes sont des combinaisons de toutes les variables ;
- **Variance $\neq$ pertinence** : la direction la plus variable n'est pas forcément la plus discriminante pour une tâche de classification (voir LDA).

Lien avec le deep learning

Un auto-encodeur *linéaire* à goulot de dimension k entraîné en erreur quadratique apprend exactement le sous-espace des k premières composantes principales : l'ACP est " l'ancêtre linéaire " des représentations apprises.

Limites de l'ACP et ouvertures

- **Linéarité** : l'ACP ne capture que des structures linéaires ; pour des variétés courbes $\rightarrow$ kernel PCA, t-SNE, UMAP, auto-encodeurs ;
- **Sensibilité à l'échelle** : standardiser les variables si elles ont des unités différentes (une variable en mètres vs en kilomètres dominerait la variance) ; inutile pour *Iris* (tout en cm) ;
- **Interprétabilité** : les composantes sont des combinaisons de toutes les variables ;
- **Variance $\neq$ pertinence** : la direction la plus variable n'est pas forcément la plus discriminante pour une tâche de classification (voir LDA).

Lien avec le deep learning

Un auto-encodeur *linéaire* à goulot de dimension k entraîné en erreur quadratique apprend exactement le sous-espace des k premières composantes principales : l'ACP est " l'ancêtre linéaire " des représentations apprises.

Limites de l'ACP et ouvertures

- **Linéarité** : l'ACP ne capture que des structures linéaires ; pour des variétés courbes $\rightarrow$ kernel PCA, t-SNE, UMAP, auto-encodeurs ;
- **Sensibilité à l'échelle** : standardiser les variables si elles ont des unités différentes (une variable en mètres vs en kilomètres dominerait la variance) ; inutile pour *Iris* (tout en cm) ;
- **Interprétabilité** : les composantes sont des combinaisons de toutes les variables ;
- **Variance $\neq$ pertinence** : la direction la plus variable n'est pas forcément la plus discriminante pour une tâche de classification (voir LDA).

Lien avec le deep learning

Un auto-encodeur *linéaire* à goulot de dimension k entraîné en erreur quadratique apprend exactement le sous-espace des k premières composantes principales : l'ACP est " l'ancêtre linéaire " des représentations apprises.

Limites de l'ACP et ouvertures

- **Linéarité** : l'ACP ne capture que des structures linéaires ; pour des variétés courbes $\rightarrow$ kernel PCA, t-SNE, UMAP, auto-encodeurs ;
- **Sensibilité à l'échelle** : standardiser les variables si elles ont des unités différentes (une variable en mètres vs en kilomètres dominerait la variance) ; inutile pour *Iris* (tout en cm) ;
- **Interprétabilité** : les composantes sont des combinaisons de toutes les variables ;
- **Variance $\neq$ pertinence** : la direction la plus variable n'est pas forcément la plus discriminante pour une tâche de classification (voir LDA).

Lien avec le deep learning

Un auto-encodeur *linéaire* à goulot de dimension k entraîné en erreur quadratique apprend exactement le sous-espace des k premières composantes principales : l'ACP est " l'ancêtre linéaire " des représentations apprises.

Exercice : ACP à la main (énoncé 1/2)

On dispose de $n = 4$ individus décrits par $d = 2$ variables :

$$X = \begin{pmatrix} 5 & 3 \\ 4 & 4 \\ 2 & 0 \\ 1 & 1 \end{pmatrix} \in \mathbb{R}^{4\times 2}.$$

On utilise la covariance $\Sigma = \frac{1}{n} X_c^T X_c$ (convention du cours).

1. Calculer le vecteur moyen $\bar{x}$ et la matrice centrée X_c.
2. Calculer Σ, puis sa décomposition spectrale $\Sigma = Q \Lambda Q^T$ avec $\lambda_1 \geq \lambda_2$.
3. Quelle part de la variance totale est expliquée par la première composante ? Combien de composantes faut-il pour dépasser 85 % ?

Exercice : ACP à la main (énoncé 2/2)

4. Calculer les scores $Z = X_c Q \in \mathbb{R}^{4\times 2}$, puis la covariance empirique des colonnes de Z. Conclure : en quoi l'ACP est-elle un changement de base qui *décorrèle* les variables ?
5. Reconstruire les données avec $k = 1$ composante : $\widehat{X} = \mathbf{1}\bar{x}^T + X_c q_1 q_1^T$. Calculer l'erreur quadratique moyenne $\frac{1}{n}\sum_i \|x_i - \hat{x}_i\|_2^2$ et la comparer à λ_2.
6. *Cas général.* Montrer que, pour des données centrées et $Q_k = [q_1|\dots|q_k]$,

$$\frac{1}{n}\sum_{i=1}^{n} \|x_i - Q_k Q_k^T x_i\|_2^2 = \sum_{j>k} \lambda_j.$$

Correction : questions 1 à 3

1. $\bar{x} = (3,2)^T, \quad X_c = \begin{pmatrix} 2 & 1 \\ 1 & 2 \\ -1 & -2 \\ -2 & -1 \end{pmatrix}.$

2. $\sum x_{i1}^2 = 10$, $\sum x_{i2}^2 = 10$, $\sum x_{i1}x_{i2} = 8$, donc

$$\Sigma = \frac{1}{4}\begin{pmatrix} 10 & 8 \\ 8 & 10 \end{pmatrix} = \begin{pmatrix} 2{,}5 & 2 \\ 2 & 2{,}5 \end{pmatrix} = 2{,}5\, I + 2\begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}.$$

$\lambda_1 = 4{,}5$, $q_1 = \frac{1}{\sqrt{2}}(1,1)^T$; $\quad \lambda_2 = 0{,}5$, $q_2 = \frac{1}{\sqrt{2}}(1,-1)^T$.

3. $\dfrac{4{,}5}{4{,}5 + 0{,}5} = 90\,\% > 85\,\%$: une seule composante suffit.

Correction : questions 1 à 3

1. $\bar{x} = (3,2)^T, \quad X_c = \begin{pmatrix} 2 & 1 \\ 1 & 2 \\ -1 & -2 \\ -2 & -1 \end{pmatrix}.$

2. $\sum x_{i1}^2 = 10$, $\sum x_{i2}^2 = 10$, $\sum x_{i1}x_{i2} = 8$, donc

$$\Sigma = \frac{1}{4}\begin{pmatrix} 10 & 8 \\ 8 & 10 \end{pmatrix} = \begin{pmatrix} 2{,}5 & 2 \\ 2 & 2{,}5 \end{pmatrix} = 2{,}5\, I + 2\begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}.$$

$\lambda_1 = 4{,}5$, $q_1 = \frac{1}{\sqrt{2}}(1,1)^T$; $\quad \lambda_2 = 0{,}5$, $q_2 = \frac{1}{\sqrt{2}}(1,-1)^T$.

3. $\dfrac{4{,}5}{4{,}5 + 0{,}5} = 90\,\% > 85\,\%$: une seule composante suffit.

Correction : questions 1 à 3

1. $\bar{x} = (3,2)^T, \quad X_c = \begin{pmatrix} 2 & 1 \\ 1 & 2 \\ -1 & -2 \\ -2 & -1 \end{pmatrix}.$

2. $\sum x_{i1}^2 = 10$, $\sum x_{i2}^2 = 10$, $\sum x_{i1}x_{i2} = 8$, donc

$$\Sigma = \frac{1}{4}\begin{pmatrix} 10 & 8 \\ 8 & 10 \end{pmatrix} = \begin{pmatrix} 2{,}5 & 2 \\ 2 & 2{,}5 \end{pmatrix} = 2{,}5\, I + 2\begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}.$$

$\lambda_1 = 4{,}5$, $q_1 = \frac{1}{\sqrt{2}}(1,1)^T$; $\quad \lambda_2 = 0{,}5$, $q_2 = \frac{1}{\sqrt{2}}(1,-1)^T$.

3. $\dfrac{4{,}5}{4{,}5 + 0{,}5} = 90\,\% > 85\,\%$: une seule composante suffit.

Correction : question 4 (décorrélation)

$$Z = X_c Q = \frac{1}{\sqrt{2}} \begin{pmatrix} 3 & 1 \\ 3 & -1 \\ -3 & 1 \\ -3 & -1 \end{pmatrix}.$$

$$\frac{1}{4}\sum_i z_{i1}^2 = \frac{1}{4}\cdot 4 \cdot \frac{9}{2} = 4{,}5, \qquad \frac{1}{4}\sum_i z_{i2}^2 = 0{,}5, \qquad \frac{1}{4}\sum_i z_{i1} z_{i2} = \frac{1}{4}\cdot\frac{3}{2}(1-1-1+1) = 0.$$

Donc

$$\mathrm{Cov}(Z) = \frac{1}{n} Z^T Z = Q^T \Sigma Q = \Lambda = \begin{pmatrix} 4{,}5 & 0 \\ 0 & 0{,}5 \end{pmatrix}.$$

- Dans la base (q_1, q_2), les nouvelles variables sont décorrélées (covariance nulle) et rangées par variance décroissante.

Correction : question 4 (décorrélation)

$$Z = X_c Q = \frac{1}{\sqrt{2}} \begin{pmatrix} 3 & 1 \\ 3 & -1 \\ -3 & 1 \\ -3 & -1 \end{pmatrix}.$$

$$\frac{1}{4}\sum_i z_{i1}^2 = \frac{1}{4}\cdot 4 \cdot \frac{9}{2} = 4{,}5, \qquad \frac{1}{4}\sum_i z_{i2}^2 = 0{,}5, \qquad \frac{1}{4}\sum_i z_{i1} z_{i2} = \frac{1}{4}\cdot\frac{3}{2}(1-1-1+1) = 0.$$

Donc

$$\mathrm{Cov}(Z) = \frac{1}{n} Z^T Z = Q^T \Sigma Q = \Lambda = \begin{pmatrix} 4{,}5 & 0 \\ 0 & 0{,}5 \end{pmatrix}.$$

- Dans la base (q_1, q_2), les nouvelles variables sont décorrélées (covariance nulle) et rangées par variance décroissante.

Correction : question 4 (décorrélation)

$$Z = X_c Q = \frac{1}{\sqrt{2}} \begin{pmatrix} 3 & 1 \\ 3 & -1 \\ -3 & 1 \\ -3 & -1 \end{pmatrix}.$$

$$\frac{1}{4}\sum_i z_{i1}^2 = \frac{1}{4}\cdot 4 \cdot \frac{9}{2} = 4{,}5, \qquad \frac{1}{4}\sum_i z_{i2}^2 = 0{,}5, \qquad \frac{1}{4}\sum_i z_{i1} z_{i2} = \frac{1}{4}\cdot\frac{3}{2}(1-1-1+1) = 0.$$

Donc

$$\mathrm{Cov}(Z) = \frac{1}{n} Z^T Z = Q^T \Sigma Q = \Lambda = \begin{pmatrix} 4{,}5 & 0 \\ 0 & 0{,}5 \end{pmatrix}.$$

- Dans la base (q_1, q_2), les nouvelles variables sont décorrélées (covariance nulle) et rangées par variance décroissante.

Correction : question 5 (reconstruction avec $k = 1$)

$\hat{x}_i = \bar{x} + z_{i1} q_1$; par exemple $z_{11} q_1 = \frac{3}{\sqrt{2}} \cdot \frac{1}{\sqrt{2}}(1,1)^T = (1{,}5\,;\, 1{,}5)^T$, d'où

$$\widehat{X} = \begin{pmatrix} 4{,}5 & 3{,}5 \\ 4{,}5 & 3{,}5 \\ 1{,}5 & 0{,}5 \\ 1{,}5 & 0{,}5 \end{pmatrix}.$$

- Résidus $x_i - \hat{x}_i = \pm(0{,}5\,;\, -0{,}5)$, de norme au carré 0,5 chacun ;
- Erreur quadratique moyenne $= 0{,}5 = \lambda_2$: l'erreur commise est exactement la variance de l'axe qu'on a jeté.

Correction : question 5 (reconstruction avec $k = 1$)

$\hat{x}_i = \bar{x} + z_{i1} q_1$; par exemple $z_{11} q_1 = \frac{3}{\sqrt{2}} \cdot \frac{1}{\sqrt{2}}(1,1)^T = (1{,}5\,;\,1{,}5)^T$, d'où

$$\widehat{X} = \begin{pmatrix} 4{,}5 & 3{,}5 \\ 4{,}5 & 3{,}5 \\ 1{,}5 & 0{,}5 \\ 1{,}5 & 0{,}5 \end{pmatrix}.$$

- Résidus $x_i - \hat{x}_i = \pm(0{,}5\,;\,-0{,}5)$, de norme au carré 0,5 chacun ;
- Erreur quadratique moyenne $= 0{,}5 = \lambda_2$: l'erreur commise est exactement la variance de l'axe qu'on a jeté.

Correction : question 6 (cas général)

Comme $(q_1, \ldots, q_d)$ est une base orthonormée :

$$x_i = \sum_{j=1}^{d} (q_j^T x_i)\, q_j, \qquad Q_k Q_k^T x_i = \sum_{j \le k} (q_j^T x_i)\, q_j.$$

Par Pythagore : $\|x_i - Q_k Q_k^T x_i\|^2 = \sum_{j>k} (q_j^T x_i)^2$. En moyennant :

$$\frac{1}{n} \sum_i \sum_{j>k} (q_j^T x_i)^2 = \sum_{j>k} q_j^T \Big(\frac{1}{n} \sum_i x_i x_i^T \Big) q_j = \sum_{j>k} q_j^T \Sigma q_j = \sum_{j>k} \lambda_j.$$

- Garder les k plus grandes valeurs propres minimise l'erreur de reconstruction : les deux formulations de l'ACP (max de variance, min d'erreur) coïncident.

Correction : question 6 (cas général)

Comme $(q_1, \ldots, q_d)$ est une base orthonormée :

$$x_i = \sum_{j=1}^{d} (q_j^T x_i)\, q_j, \qquad Q_k Q_k^T x_i = \sum_{j \le k} (q_j^T x_i)\, q_j.$$

Par Pythagore : $\|x_i - Q_k Q_k^T x_i\|^2 = \sum_{j>k} (q_j^T x_i)^2$. En moyennant :

$$\frac{1}{n} \sum_i \sum_{j>k} (q_j^T x_i)^2 = \sum_{j>k} q_j^T \Big(\frac{1}{n} \sum_i x_i x_i^T \Big) q_j = \sum_{j>k} q_j^T \Sigma q_j = \sum_{j>k} \lambda_j.$$

- Garder les k plus grandes valeurs propres minimise l'erreur de reconstruction : les deux formulations de l'ACP (max de variance, min d'erreur) coïncident.

Correction : question 6 (cas général)

Comme $(q_1, \ldots, q_d)$ est une base orthonormée :

$$x_i = \sum_{j=1}^{d} (q_j^T x_i)\, q_j, \qquad Q_k Q_k^T x_i = \sum_{j \le k} (q_j^T x_i)\, q_j.$$

Par Pythagore : $\|x_i - Q_k Q_k^T x_i\|^2 = \sum_{j>k} (q_j^T x_i)^2$. En moyennant :

$$\frac{1}{n} \sum_i \sum_{j>k} (q_j^T x_i)^2 = \sum_{j>k} q_j^T \Big(\frac{1}{n} \sum_i x_i x_i^T \Big) q_j = \sum_{j>k} q_j^T \Sigma q_j = \sum_{j>k} \lambda_j.$$

- Garder les k plus grandes valeurs propres minimise l'erreur de reconstruction : les deux formulations de l'ACP (max de variance, min d'erreur) coïncident.

Ce qu'il faut retenir du chapitre 1

Concepts clés

- Données $=$ vecteurs ; modèles $=$ transformations linéaires (matrices) ;
- Base $=$ représentation ; changement de base $A' = P^{-1}AP$;
- $\det A \neq 0 \iff A$ inversible ; $|\det| =$ facteur de volume ;
- $Av = \lambda v$: directions invariantes ; spectre $\Rightarrow$ dynamique de A^t.

Pour l'IA

- Théorème spectral : covariances, noyaux, hessiennes ;
- ACP $=$ vecteurs propres de la covariance $=$ meilleure projection ;
- PageRank, RNN, flows : les valeurs propres sont partout.